

Systems Requirements Document (SRD)

1. Introduction

1.1 Purpose :

The purpose of this document is to outline the system requirements for **Smart Save**, a mobile and web application designed for young professionals to help them manage their finances through automated savings, spending tracking, and AI-driven budgeting insights. This document serves as a guide for developers, designers, stakeholders, and testers to ensure the delivered product meets business and user needs.

1.2 Problem :

Young professionals often face challenges in saving consistently, understanding their spending habits, and using budgeting tools that may be too complex or inaccessible. This leads to poor financial wellness and missed opportunities for building savings. There is a need for a simple, automated, and personalized solution that encourages positive financial behaviors.

1.3 Document Conventions:

- Must / Shall: Mandatory requirements
- Should: Recommended but not mandatory
- May / Optional: Desirable features but not required
- UI: User Interface
- AI: Artificial Intelligence

1.4 Project Scope:

Smart Save will provide users with:

- Automated savings capabilities linked to their bank accounts
- Real-time spending tracking
- Budget creation and alerts based on personalized insights
- AI-generated recommendations to improve financial habits
- Secure and user-friendly web and mobile applications

1.5 Intended Audience:

- Stakeholders and Sponsor(s)
- Product Manager/Product Owner
- Young Professionals

- Other Users

2. Overall Description

2.1 Product Perspective :

Smart Save is a standalone FinTech application designed to integrate with users' bank APIs for financial data. It is positioned as a personal finance assistant accessible on mobile (iOS/Android) and web browsers. The product leverages AI for personalized insights and automation.

2.2 Use Cases and Characteristics :

- Users link bank accounts securely to import transactions
- Users view categorized spending and savings overviews
- Users set and modify budgets and savings goals
- The system automatically transfers savings amounts based on AI recommendations
- The app sends notifications and nudges to promote financial wellness
- Users can customize preferences for notifications, budgeting categories, and saving rules

2.3 Evaluation Status States:

- Requirements will be reviewed iteratively with stakeholders every two weeks during the project
- User acceptance testing (UAT) to validate functional requirements before deployment
- AI insights will be evaluated with a small user group for accuracy and usability

2.4 Operating Environment :

- Mobile: iOS 13+ and Android 9+
- Web: Modern browsers (Chrome, Firefox, Safari, Edge)
- Secure cloud backend with API integrations to financial institutions
- Data storage compliant with GDPR and other relevant data protection laws.

2.5 Design and Implementation Constraints :

- Must comply with financial data security standards (e.g., PCI DSS, GDPR)
- Must maintain secure handling of user credentials and sensitive information
- AI recommendations must explain reasoning behind nudges to build trust

- Real-time syncing depends on third-party banking APIs availability and latency

2.6 Assumptions and Dependencies :

- Users have active bank accounts compatible with API integrations
- Network connectivity is available for syncing financial data
- Users consent to data access and automated transaction scheduling
- AI module integrates seamlessly with core application backend

3. System Features

3.1 System Requirements

Feature	Description	Priority
Secure User Authentication	Allow users to sign up, log in, and manage their profiles securely using multi-factor authentication.	Must
Account Linking	Enable users to securely link their bank accounts via APIs for transaction import and syncing.	Must
Spending Tracking	Automatically categorize and display users' spending by category and time period.	Must
Budget Setting	Allow users to create budgets with alerts when nearing or exceeding limits.	Must
Automated Savings Transfer	Automatically move predefined amounts to a savings account based on AI-generated recommendations.	Must
AI Insights and Nudge Engine	Provide personalized insights and notifications to improve financial habits.	Should
Data Visualization	Dashboard displaying spending, saving trends, and budget progress with charts and summaries.	Should
Notification Preferences	Allow users to customize notification types	Should

	and frequency.	
--	----------------	--

3.2 User Requirements:

- Users must be able to view a clear summary of their financial position at any time.
- Users must be able to adjust savings goals manually.
- Users want personalized financial advice that is easy to understand.
- Users need assurance that their data is fully protected with best practices.
- Users expect fast and intuitive navigation on both mobile and web platforms.

4. Use Cases

Use-Case ID	Name	Description
UC-01	User Registration and Login	User creates an account or logs in with multi-factor authentication.
UC-02	Link Bank Account	User securely connect bank accounts for data synchronization.
UC-03	View Spending Overview	User reviews categorized transactions and spending patterns.
UC-04	Set Budget	User set monthly budgets with optional categories.
UC-05	Receive AI Savings Recommendation	System analyzes spending to suggest saving transfers.
UC-06	Automated Savings Transfer	System initiates transfers to savings based on rules and AI insights.
UC-07	Manage Notification	User sets preferences for alerts and nudges.

4.2 Use-Cases Expansion

UC-01: User Registration and Login

Primary Actor: User

Preconditions: User has access to internet and device.

Flow:

1. User opens the app/website.
2. User selects “Register” or “Login.”
3. User enters email and password (or opts for social login).
4. User completes multi-factor authentication (e.g., OTP).
5. System authenticates and directs the user to the dashboard.
6. User can reset password(s) via email link.

Acceptance Criteria:

- ☐ Users must be able to register with valid email and password.
- ☐ MFA is enforced for login.
- ☐ Password reset functionality works correctly.
- ☐ Error messages appear for invalid inputs.

UC-02: Link Bank Account

Primary Actor: User

Preconditions: User logged in successfully.

Flow:

1. User chooses to link bank account.
2. System displays a list of supported banks.
3. User selects bank and inputs credentials securely.
4. System verifies credentials via API and retrieves transaction data.
5. Confirmation of successful linking shown to the user.

Acceptance Criteria:

- ☐ Only supported banks are shown.
- ☐ Credentials are transmitted securely.
- ☐ User receive confirmation on success or failure.
- ☐ Linked accounts show updated transactions.

UC-03: View Spending Overview

Primary Actor: User

Preconditions: Bank account linked and transactions synced.

Flow:

1. User navigates to spending overview/dashboard.
2. System shows categorized spending by time periods (weekly, monthly).
3. User can drill down into categories to see detailed transactions.
4. System provides visual charts and summaries.

Acceptance Criteria:

- ☐ Spending data refreshes after sync.
- ☐ Categories are accurate and consistent.
- ☐ Visualizations load within 3 seconds.

UC-04: Set Budget

Primary Actor: User

Preconditions: User logged in with at least one linked account.

Flow:

1. User selects "Create Budget."
2. User selects categories and defines budget limits.
3. User saves budget.
4. System tracks spending against budget and triggers alerts.

Acceptance Criteria:

- ☐ Budgets can be created, edited, and deleted.
- ☐ Alerts are sent when spending reaches 80% and 100% of budgets.
- ☐ Budgets persist across sessions.

UC-05: Receive AI Saving Recommendations

Primary Actor: System (AI Engine)

Preconditions: Historical spending data available.

Flow:

1. AI analyzes user spending and income patterns.
2. AI generates recommended savings amounts.
3. System presents recommendations to users with explanations.
4. Users accept or modify recommendations.

Acceptance Criteria:

- ☐ Recommendations update weekly.
- ☐ Explanations are understandable and transparent.
- ☐ Users can modify or decline recommendations.

UC-06: Automated Savings Transfer

Primary Actor: System

Preconditions: User has accepted AI saving recommendations.

Flow:

1. System schedules savings transfer based on user preference.
2. Transfer initiated via linked bank accounts automatically.
3. User receives confirmation of transfer.
4. Transfer records updated in user transaction history.

Acceptance Criteria:

- ☐ Transfers happen automatically as scheduled.
- ☐ Failures generate notifications for users.
- ☐ Transfer history is accurate and accessible.

UC-07: Manage Notifications

Primary Actor: User

Preconditions: User logged in.

Flow:

1. User accesses notification settings.
2. User selects preferred notification types and frequency (email, push, SMS).
3. User saves preferences.
4. System respects user preferences for all communications.

Acceptance Criteria:

- ☐ Changes in notification preferences update immediately.
- ☐ Notifications are sent only for selected types.
- ☐ Users can opt-out completely.

5. External Interface Requirements

5.1 User Interface:

- Mobile Apps: Clean, responsive UI optimized for small screens; intuitive navigation with tabs/dashboards.
- Web Interface: Responsive design compatible with common browsers; detailed dashboards with drag/drop budget adjustments.
- Accessibility: Support for screen readers, high-contrast mode, and keyboard navigation.

5.2 Software Interfaces:

- Integration with banking APIs for transaction data retrieval.
- Backend API for syncing user data and AI recommendation engine.
- Authentication interface supporting OAuth2 or similar secure protocols.

5.3 Communication Interfaces:

- Push notifications via mobile OS (iOS/Android) services.
- Email notifications for reports and critical alerts.
- Encrypted data transmission using HTTPS/TLS standards.

6. Non-Functional Requirements

Requirements Category	Description
Security	Data encrypted in transit and at rest; compliance with GDPR standards.
Performance	Application load times under 3 seconds; transaction syncing within 30 seconds of bank updates.
Usability	Intuitive design with metrics for ease of use; supports accessibility guidelines.
Reliability	99.9% uptime with failover mechanisms; transaction accuracy maintained
Scalability	Supports scaling to thousands of users without

	performance degradation.
Maintainability	Modular codebase with thorough documentation and automated testing coverage.
Compatibility	Support for the latest two major versions of iOS, Android, and web browsers.